За пределами сжатия контекста: Состав состояния и недавняя история программирующего агента

Alex Markov-Buturskiy

Аннотация

Структурированное состояние обещает ограничить контекст агента, заменив растущую историю взаимодействия компактной памятью. Однако размер промпта не определяет, сколько работы потребуется для решения задачи. Мы исследуем, какие сведения должно сохранять состояние программирующего агента и кто должен их записывать. Сравниваются память, поддерживаемая моделью, гибридное состояние с автоматическими записями среды исполнения и сохранение недавней истории; отдельно проверяются схема и протокол обновления. При замене истории состоянием модели пиковый контекст на общих успехах сокращается до 0.47 от контрольного, но стоимость возрастает в 2.57 раза, а общая успешность снижается с 61.3% до 34.9%. Поисковое сравнение состава гибридной памяти показывает преимущество записей изменений и команд перед одним списком файлов, без устойчивого улучшения от каждого следующего компонента. В повторных измерениях сохранение недавней истории повышает успешность состояния модели до 59.0%, гибридной памяти — до 48.3% при 51.7% у её собственного контроля. Проверенные изменения схемы и inline-протокол такого восстановления не дают. Стоимость на успех при сохранении истории снижается в первой реализации, но превышает контрольную во второй. Результаты показывают пользу компактной памяти вместе с границами её применения: содержание автоматических записей и доступ к исходным взаимодействиям требуют отдельных решений, а экономию контекста следует оценивать вместе с качеством и стоимостью выполнения.

1 Введение

Программирующий агент узнаёт задачу через действия. Команда обнаруживает зависимость, тест опровергает исправление, сообщение компилятора связывает ошибку с конкретной правкой. Последующие решения зависят от того, какие сведения об этих действиях остались доступны. Если история взаимодействия растёт на каждом шаге, вместе с ней растёт и объём входа модели. Если её сжать, приходится решать, чем заменить удалённые наблюдения.

SKILL.state предлагает хранить изменяемое структурированное состояние вместо полного транскрипта [Badhe et al., 2026]. Привлекательность этого решения состоит в отделении текущей рабочей памяти от накопленной истории. При ограниченном размере состояния контекст не обязан расти с каждым действием. Но ограничение размера представления само по себе не ограничивает объём информации, необходимой для решения задачи. Краткая запись о неудаче может не сохранить сообщение об ошибке; список изменённых файлов не сообщает, что именно было изменено и почему.

Из этого возникает инженерный вопрос: какую работу по ведению памяти следует поручить модели? Гипотезы и планы требуют интерпретации. Выполненная команда и её наблюдённый результат уже известны среде исполнения. Повторная генерация этих сведений моделью расходует выходные токены и делает их сохранность зависимой от обновлений семантического состояния. Разделение обязанностей позволяет хранить наблюдения автоматически, оставляя модели рассуждение об их значении. Мы называем такое представление гибридным состоянием.

В работе проверяются возможности этого разделения на задачах программирующего агента. Сначала оценивается цена замены истории состоянием модели. Затем сравнивается состав автоматической памяти: от ссылок на файлы до записей изменений, команд, диагностик и результатов тестов. Следующие сравнения проверяют, устраняют ли оставшееся отставание недавняя история, адаптация схемы или другой протокол обновления. Качество рассматривается вместе с ресурсами, поскольку меньший промпт может сочетаться с большей генерацией и более высокой стоимостью успешного выполнения.

Результаты дают различающиеся ответы на эти вопросы. Компактное состояние действительно уменьшает пиковый контекст, но без истории теряет успешность. В поисковом сравнении более содержательные записи среды исполнения заметно лучше бедного списка файлов; дальнейшее обогащение не даёт последовательного улучшения. В повторных измерениях наиболее выраженное восстановление успешности связано с недавней историей. Испытанные схема и inline-протокол не обеспечивают аналогичного результата. При этом экономический эффект сохранения истории различается между реализациями.

Вклад работы состоит в эмпирическом разделении решений, которые легко объединить под названием «память агента»: кто записывает сведения, что входит в состояние, какие взаимодействия остаются в исходном виде и как оплачивается дальнейшая работа. Это исследование переноса идеи SKILL.state в другой агентный стек, а не повторение исходных экспериментов. Его результаты характеризуют испытанные реализации и уточняют, какие изменения частично компенсируют их ограничения.

2 Подходы к памяти и исследовательские вопросы

История и состояние по-разному сохраняют информацию. История оставляет записи взаимодействия в исходной последовательности. Состояние преобразует выбранные сведения в описание текущей задачи. Поле состояния может сообщать, что тест завершился неудачно, но не сохранять полный вывод теста и предшествующие правки. В то же время наличие исходной последовательности не гарантирует, что модель правильно ею воспользуется.

Различается и способ формирования состояния. Модель может поддерживать гипотезы, план и описание незавершённой работы. Среда исполнения может записывать непосредственно наблюдаемые события: выполненные команды и их результаты. Представление, объединяющее эти источники, далее называется *гибридным состоянием*. Автоматическая запись уменьшает необходимость заново описывать события, но сама по себе не даёт ни их интерпретации, ни плана следующего действия.

Опубликованный контракт SKILL.state объединяет изменение состояния и действие в одном ответе модели [Badhe et al., 2026]. Отдельный инструмент обновления меняет этот порядок: ведение памяти становится инструментальной операцией внутри цикла агента. Вариант с inline-обновлением проверяет совместную передачу изменения и действия, а сравнение схем — организацию записываемого содержания. Эти изменения относятся к реализации переноса, но не устраняют все отличия от опубликованной системы.

Исследование начинается с сопоставления сжатия контекста и стоимости решения. Далее проверяются состав автоматических записей, сохранение недавней истории и изменения схемы с протоколом. Эти сравнения различают содержимое памяти, источник её обновлений и доступность исходных взаимодействий. Для ответа нужны сопоставления с собственным контролем каждого подхода. В частности, добавление истории к состоянию проверяет пользу этой истории внутри системы со состоянием, но не позволяет выделить пользу самого блока состояния.

3 Контекст агента и протоколы обновления

Обозначим через P постоянные инструкции задачи, через o_t текущее наблюдение, а через $H_{<t}$ — предшествующие записи взаимодействия. Под записями понимаются доступные сообщения и

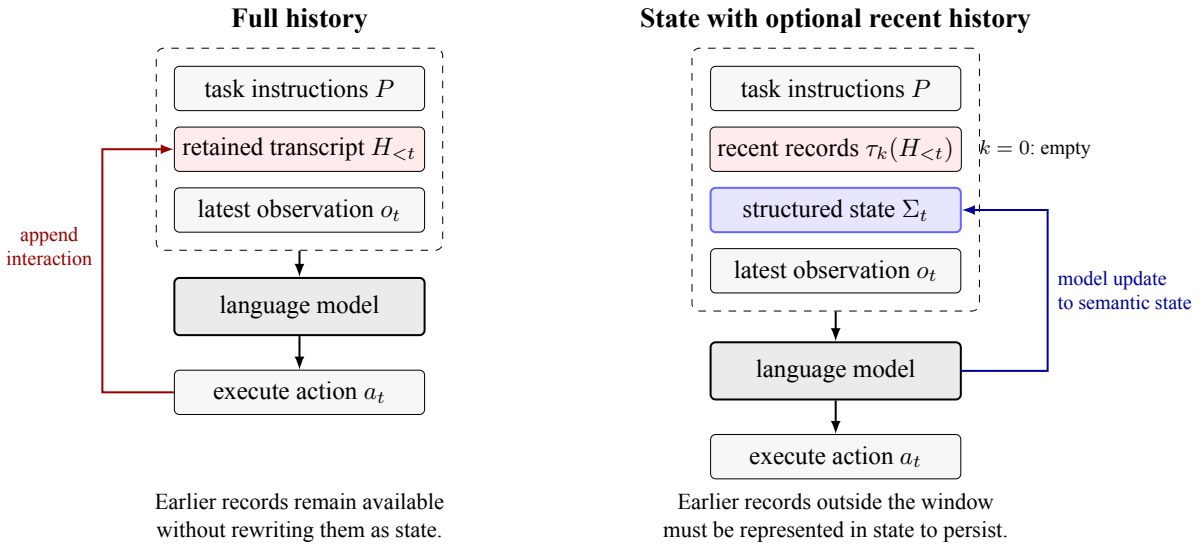


Figure 1: Context policies. State replaces the older transcript, while the retention parameter k controls the recent suffix. The arrows show information flow, not the number of model calls.

результаты инструментов; доступ к скрытым рассуждениям модели не предполагается. При сохранении полной истории контекст имеет вид

$$C_t^{\text{full}} = [P, H_{<t}, o_t]. \tag{1}$$

Если прошлую историю заменяет структурированное состояние Σ_t, то

$$C_t^{\text{state},k} = [P, \Sigma_t, \tau_k(H_{<t}), o_t], \qquad \tau_0(H) = \varnothing, \tag{2}$$

где τ_k выбирает недавнюю историю согласно настройке k. В начале задачи окно ограничено уже имеющимися записями. Полная история служит отдельным контрольным подходом, а не вариантом состояния с неограниченным окном. Источники информации в обоих случаях показаны на рис. 1.

3.1 Кто записывает состояние

В первом подходе модель сама поддерживает семантическое состояние S_t. Гибридная память разделяет это содержание и записи среды исполнения:

$$\Sigma_t = (S_t, V_t), \qquad S_{t+1} = \text{Apply}(S_t, \Delta S_t), \qquad V_{t+1} = \text{Record}(V_t, a_t, r_t). \tag{3}$$

Компонент V_t пополняется на основании наблюдаемых событий инструментов r_t. Модель не может изменять его через обновление семантического состояния. Однако такое разделение прав записи само по себе не гарантирует ни полноты журнала, ни правильности интерпретации каждого события. Состав V варьируется от ссылок на файлы до записей изменений, команд с их выводом, диагностик, сведений о среде, результатов тестов и неудачных попыток. Это описание групп сохраняемых сведений, а не гарантия их полного извлечения. Например, распознавание тестового результата из вывода инструмента может зависеть от эвристики. Недавняя история может добавляться к обоим типам состояния независимо от того, кто его записывает.

3.2 Как обновление связано с действием

При отдельном инструменте обновления модель передаёт изменение состояния через интерфейс вызова инструмента. При inline-протоколе она формирует единый ответ $\{\Delta S_t, a_t\}$, содержащий изменение состояния и действие. Такая форма соответствует опубликованному контракту

SKILL.state [Badhe et al., 2026]. Протоколы различаются тем, как ведение памяти связано с обращениями к среде: шаг с отдельным инструментом не обязательно укладывается в один вызов модели.

Алгоритм 1 задаёт выбор контекста и разделение прав записи. Процедура PolicyStep не фиксирует сериализацию сообщений и количество промежуточных вызовов модели; при полной истории она выдаёт действие, не изменяя S. Алгоритм 2 описывает совместный контракт обновления и действия: при недопустимом изменении действие из отвергнутого ответа не выполняется. Это описание сравниваемого контракта, а не восстановленная трасса исполнения и не подтверждение соблюдения всех его деталей каждой реализацией. Фиксированное число повторных попыток и автоматический возврат к полной истории не предполагаются.

Algorithm 1 Context selection and state ownership

Require: Instructions P, policy π, retention setting k, update protocol q, initial observation o

Initialize history H, semantic state S, runtime record V
while the task has not terminated **do**
    if $\pi =$ FullHistory **then**
        $C \leftarrow [P, H, o]$
    else
        $\Sigma \leftarrow S$ **or** (S, V) according to state ownership
        $C \leftarrow [P, \Sigma, \tau_k(H), o]$
    end if
    $(S', a) \leftarrow \text{PolicyStep}(C, S, \pi, q)$
    $r \leftarrow \text{Execute}(a)$; $S \leftarrow S'$
    if hybrid memory is enabled **then**
        $V \leftarrow \text{Record}(V, a, r)$
    end if
    Append a record of (o, a) to H; $o \leftarrow r$
end while

Algorithm 2 Inline patch–action contract

Require: Current context C and semantic state S

$(\Delta S, a) \leftarrow \text{Model}(C)$
if $\text{Valid}(\Delta S, S)$ **then**
    return $(\text{Apply}(S, \Delta S), a)$
else
    Keep S unchanged and return a validation observation
    Do not execute a from the rejected envelope
end if

3.3 Размер контекста и стоимость выполнения задачи

Если размер отдельных записей и число вызовов модели на одно действие ограничены, полная история даёт рост контекста порядка t, а накопленного входа за T действий — порядка T^2. Размер контекста со структурированным состоянием не зависит от номера действия только при ограничениях на само состояние, текущее наблюдение и записи недавней истории. При этих допущениях и постоянном k накопленный вход растёт как $O(T)$. Ни одна из оценок не определяет, сколько действий агенту потребуется для завершения задачи.

Пусть U, K и O — суммарные количества некешированных входных, прочитанных из кеша и выходных токенов. Для тарифов α, γ и β нормированная стоимость равна

$$D = \alpha U + \gamma K + \beta O. \tag{4}$$

По одному максимальному размеру контекста нельзя восстановить эти суммы. Поддержание состояния может увеличивать выход модели, а изменение контекста — влиять на повторное использование кеша. Это возможные объяснения, требующие проверки, а не следствия оценки размера контекста. Поэтому в эксперименте отдельно рассматриваются успешность, размер контекста, категории токенов и стоимость.

4 Методология

4.1 Агент, задачи и сравниваемые варианты

Используются модель `zai/glm-5.3-flash`, программирующий агент Pi версии 0.85.1 и среда оценки Harbor семейства 0.22. Задачи взяты из Terminal-Bench 2.0, который предоставляет исполняемые окружения и специализированные проверки результата [Merrill et al., 2026]. Агент взаимодействует со средой через инструменты работы с кодом и терминалом. Успех определяется бинарным результатом проверки задачи; зарегистрированное ограничение составляет 200 ходов.

Анализируемое подмножество включает 87 задач; ещё две задачи исключены из-за невозможности инициализировать их окружения до начала работы агента. Для состояния модели сравниваются четыре условия с тремя повторениями на задачу. Поисковое сравнение состава автоматической памяти включает четыре конфигурации и собственный контроль с одним повторением на том же наборе задач. Итоговое сравнение гибридной памяти включает восемь условий с двумя повторениями. Результаты контроля и двух наиболее полных наборов записей из поискового сравнения входят также в первое повторение итогового; эти сравнения не являются независимыми воспроизведениями. В каждом сравнении есть собственный контроль с полной историей. Единица наблюдения — выполнение задачи при заданном условии и повторении; сопоставления выполняются по одной и той же задаче и повторению. Контроли не объединяются, а повторные выполнения не считаются дополнительными задачами.

Поисковое сравнение состава использует накопительные наборы автоматических записей без недавней истории, при фиксированных схеме и отдельном инструменте обновления. Некоторые переходы добавляют сразу несколько компонентов. Конфигурации с более низкой предварительной успешностью не продолжили измерять; результаты характеризуют исследовательский отбор, а не заранее завершённую повторную проверку всей последовательности.

Варианты состояния модели используют отдельный инструмент обновления и настройки недавней истории $k = 0, 2, 6$. Гибридный опорный вариант объединяет записи среды исполнения, включая неудачные попытки, с семантическим состоянием модели. Его модификации исключают сведения о неудачах, добавляют недавнюю историю либо образуют сравнение tool/inline-протоколов с исходной/адаптированной схемами. Опорный гибридный вариант общий для этих сопоставлений. Схема адаптировалась с учётом опыта на том же наборе задач; поэтому сравнение имеет поисковый характер и не является проверкой на независимой выборке.

Параметр k обозначает настройку сохранения истории. Доступные материалы не позволяют единообразно установить, соответствует ли его единица отдельному сообщению или полному обмену с инструментом. Поэтому равные значения k не трактуются как подтверждение одинаковой сериализации контекста. Формальная модель описывает источники информации и порядок обновления, а не хранение скрытых рассуждений или точный формат API.

4.2 Показатели успешности и ресурсов

Пусть Y_{iar} — бинарный результат для условия a, задачи i и повторения r. Для n задач и R повторений доля успешных выполнений равна

$$\widehat{s}_a = \frac{1}{nR} \sum_{i=1}^{n} \sum_{r=1}^{R} Y_{iar}. \tag{5}$$

Разности успешности приводятся в процентных пунктах относительно явно указанного контроля того же сравнения.

Пиковый контекст — максимальный размер входа отдельного вызова модели. Он отличается от суммарных некешированных входных, прочитанных из кеша и выходных токенов за всё выполнение. Для состояния модели применяется фиксированный условный тариф:

$$D = 10^{-6}(0.15U + 0.03K + 0.50O), \tag{6}$$

где U, K и O соответствуют этим трём категориям токенов. Стоимость выражена в долларах США по выбранному тарифу, а не по выставленным провайдером счетам. Записанная стоимость отдельных выполнений поискового сравнения состава сверена с тем же тарифом с учётом округления. Для итоговых гибридных сравнений используются медианы и округлённые суммы из отчёта, также трактуемые как нормированные затраты, а не счета провайдера. Их формулу тарификации нельзя независимо проверить по доступным агрегатам; пиковый контекст не выводится из суммарных входных токенов.

Приводятся медианная стоимость выполнения и стоимость на наблюдаемый успех $\sum D / \sum Y$ по всем включённым выполнениям, в том числе неудачным. Обратная величина — число успехов на доллар. Для успехов на миллион токенов знаменатель включает накопленные входные, кешированные и выходные токены. Эти показатели характеризуют учтённую нагрузку, без затрат на исключённые попытки и не вошедшие в данные повторы. Они не оценивают стоимость стратегии повторять задачу до первого успеха.

Отдельно рассматриваются выполнения, в которых оба подхода успешно справились с одной задачей в одном повторении. Для каждого ресурса вычисляется медиана отношений вариант/контроль внутри таких сопоставлений. Подмножество зависит от сравниваемого варианта. Этот показатель описывает расход на общих успехах, тогда как стоимость на успех учитывает также неудачи и различия в успешности. Медиана отношений не заменяется отношением отдельных медиан. Для поискового сравнения состава дополнительно фиксируется одно подмножество задач, успешных при всех конфигурациях и контроле. Сопоставление этих способов отбора проверяет чувствительность условных стоимостных выводов к составу задач.

4.3 Статистический анализ

В таблицах успешности используются двусторонние точные p-значения критерия Мак-Немара для указанных сопоставлений. Для отношений стоимости на общих успехах применяется двусторонний точный критерий знаков относительно единицы, с исключением равенств. Это поисковые проверки без поправки на множественность, предполагающие независимость сопоставленных наблюдений. Повторные задачи и возможные зависимости между выполнениями ограничивают такую интерпретацию: проверки не учитывают кластеризацию по задачам и не контролируют совокупную вероятность ошибки во всём семействе сравнений.

Для состояния модели дополнительно вычисляются поточечные бутстрэп-интервалы разности успешности. Перевыбираются идентификаторы задач вместе со всеми условиями и повторениями; используются 100 000 перевыборок и seed 20260922. Границы 95%-го интервала задаются процентилями. Процедура сохраняет наблюдаемую внутригрупповую структуру, но не доказывает, что сам бенчмарк представляет случайную выборку задач. В итоговых агрегатах гибридной памяти нет межусловных результатов на уровне каждой задачи, необходимых для такого же анализа. Малые различия и незначимые проверки не интерпретируются как эквивалентность.

Дополнительный анализ длины траектории разделяет задачи по медианному числу ходов T у контроля с полной историей, с границей $T = 11$. Состав подмножеств фиксирован для всех вариантов состояния. Такое апостериорное разбиение описывает короткие и длинные контрольные траектории, но не оценивает причинное влияние их длины.

4.4 Материалы анализа

Результаты состояния модели пересчитываются из опубликованных записей отдельных выполнений с сохранением исходных правил включения. Для поискового сравнения состава доступны отдельные записи с исходами и ресурсами; полная последовательность компонентных вариантов ими не покрывается. Для итогового сравнения гибридной памяти используются окончательные сводки по условиям и числа несовпадающих исходов в сопоставлениях. Последние достаточны для приведённых сравнений успешности, но не для восстановления траекторий. Доступные материалы и определения производных показателей описаны в приложении.

5 Сжатие контекста и цена решения задачи

5.1 Компактное представление не сохраняет результат автоматически

Состояние модели без истории выполняет основную функцию сжатия: на задачах, успешно завершённых и им, и контролем, медианное отношение пикового контекста составляет 0.47. Однако общая успешность снижается с 61.3% до 34.9%. Бутстрэп-интервал разности равен $[-34.1, -18.8]$ процентного пункта. Сокращение входа отдельного вызова здесь не сопровождается сохранением способности завершать задачи.

Стоимость тех же общих успехов показывает другую сторону компромисса. Её медианное отношение к полной истории равно 2.57, а отношение выходных токенов — 4.55 (табл. 1). На этом подмножестве агент получает меньший пиковый контекст, но суммарно генерирует больше и обходится дороже по фиксированному тарифу.

Table 1: Resource ratios on shared successes: median of matched condition/full-history ratios. Peak prompt is a per-call maximum; token columns are cumulative. The exact sign test concerns cost ratios. Each condition has its own shared-success subset.

Approach	Cost	Peak prompt	Input	Output	Cached	p
Model-written state	2.57×	0.47×	1.86×	4.55×	0.89×	1.4×10^{-12}
State + history ($k = 2$)	1.39×	0.60×	1.62×	2.08×	0.40×	4.7×10^{-4}
State + history ($k = 6$)	1.15×	0.81×	1.96×	1.36×	0.21×	0.036

5.2 Из чего складываются издержки

Таблица 2 показывает, что различие не ограничивается размером памяти. По всем включённым выполнениям состояние без истории имеет больше ходов и больший медианный выход, чем полная история. Обновления памяти тоже должны быть сгенерированы; дополнительные действия требуют новых обращений к модели. При этом длинный вход не обязательно целиком оплачивается по тарифу некешированных токенов. Полная история и изменяемое состояние дают разные возможности повторного использования префикса.

Table 2: Median recorded turns T, per-execution peak prompt, and cumulative token use for model-written memory. Token counts are in thousands; input excludes cache reads. Peak is the median of execution maxima. Token columns are separate marginal medians, not an additive median total.

Approach	Median T	Peak prompt (ktok)	Input (ktok)	Output (ktok)	Cached (ktok)
Full history	13	16.1	28.8	16.4	87.4
Model-written state	25	4.0	24.9	37.4	36.7
State + history ($k = 2$)	14	6.2	30.3	20.1	19.1
State + history ($k = 6$)	11	11.3	53.3	16.6	14.8

Эти соображения объясняют, почему сжатие нужно оценивать по всему выполнению. Счётчики подтверждают различия в ходах, выходе и использовании кеша, но не позволяют определить, какая доля издержек вызвана повторным изучением среды, текстом обновлений или иными изменениями поведения. Проверяемый результат уже достаточен для следующего шага: уменьшить контекст оказалось недостаточно; требуется исследовать содержание сохраняемой памяти.

Стоимость на наблюдаемый успех с учётом неудач составляет $0.1630 для состояния без истории против $0.1116 для полной истории. Это другой показатель, чем условное отношение 2.57: он учитывает весь сохранённый в данных объём работы. Дальнейшие сравнения приводят успешность вместе со стоимостью, чтобы улучшение одного показателя не скрывало ухудшения другого.

6 Какие сведения должен сохранять рантайм

6.1 Наблюдения среды и семантическая память

Среда исполнения уже получает результаты инструментов. Ей не требуется вызывать модель, чтобы сохранить команду и доступный вывод. Модель, напротив, нужна для выбора гипотезы, объяснения ошибки и планирования работы. Гибридное состояние разделяет эти обязанности: автоматические записи V хранят захваченные наблюдения, семантическое состояние S — их интерпретацию и незавершённую работу (рис. 2).

Такое разделение предлагает конкретный способ уменьшить издержки ведения памяти. Сведения, сохранённые средой исполнения, не требуется заново генерировать в семантическом блоке; модель также не может стереть их своим обновлением S. Но объём и содержание V остаются проектными решениями. Один список файлов не заменяет результатов команд, а накопление всех доступных записей увеличивает вход. Поэтому проверять нужно не только наличие автоматической памяти, но и её состав.

6.2 Состав автоматических записей: поисковое сравнение

Сравниваются накопительные наборы сведений при неизменных настройках схемы, отдельном инструменте обновления и отсутствии недавней истории. Минимальная конфигурация сохраняет ссылки на затронутые файлы. Следующая добавляет записи изменений и команд с их выводом. Более полная конфигурация включает также диагностики, сведения о среде и результаты тестов; последняя добавляет записи неудачных попыток. Промежуточные компоненты не исследованы по отдельности, поэтому каждый измеренный переход характеризует весь добавленный набор.

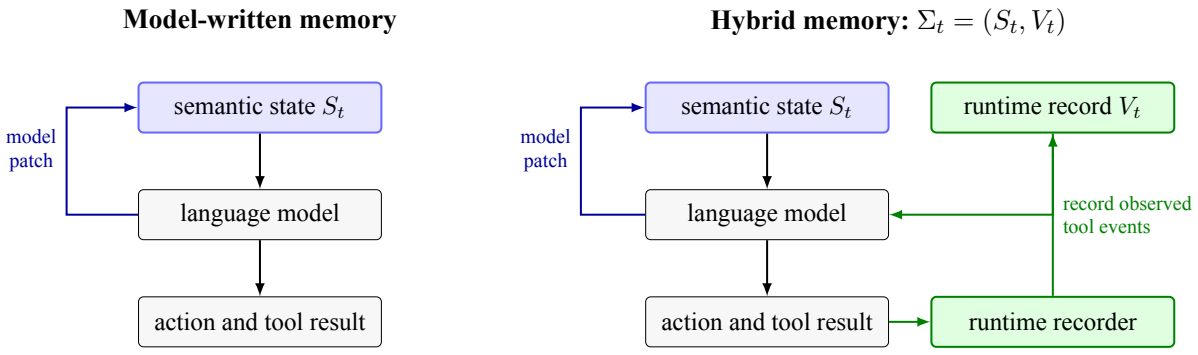


Figure 2: State ownership. Hybrid memory retains model-written semantic content and adds runtime-derived records. Recording an event does not establish that an extracted interpretation is correct or that all relevant events were captured.

Table 3: Exploratory comparison of cumulative runtime records, without recent history. Output and cost are medians over all retained executions. Solved/$ uses aggregate cost, including failures. Exploratory first-repeat subset; the control and richer configurations also contribute to the final comparison.

Runtime-owned information	Solved rate (%)	Median output (ktok)	Median cost ($)	Solved/$
Full history	55.2	16.4	0.01139	22.39
File references	28.7	23.2	0.01406	12.87
Files, changes, commands	40.2	16.5	0.01016	20.79
+ diagnostics, environment, tests	42.5	19.1	0.01316	20.92
+ failed-attempt records	41.4	18.6	0.01196	21.19

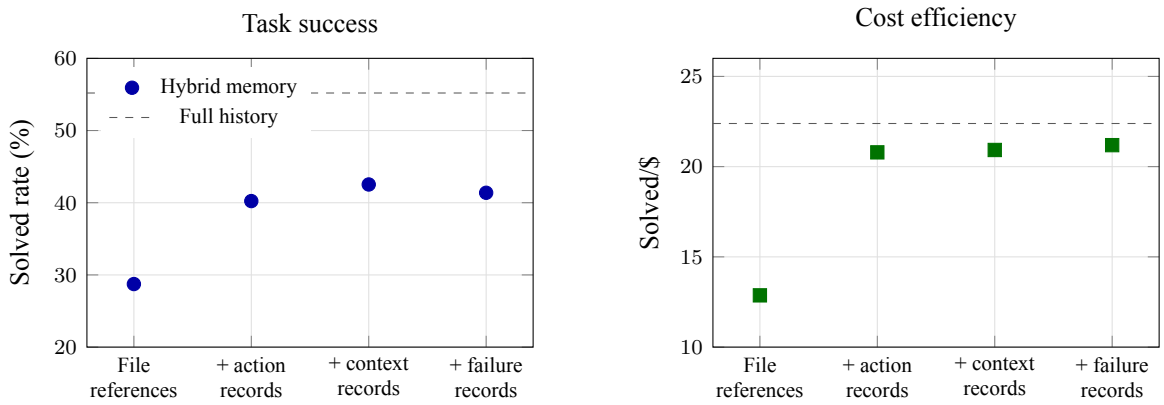


Figure 3: Exploratory comparison of cumulative runtime-owned information. Each position adds records to the preceding configuration: action records cover file changes and commands; context records cover diagnostics, environment, and tests. Efficiency includes the cost of unsuccessful episodes. Dashed lines show the same full-history reference. These descriptive observations share their richer configurations and reference with the first repeat of the final comparison; they do not isolate individual components or establish a monotonic benefit.

Переход от ссылок на файлы к записям изменений и команд сопровождается ростом успешности с 28.7% до 40.2% и снижением медианного выхода с 23.2 до 16.5 тысяч токенов. Это наиболее выраженное изменение в данном сравнении. Дополнение диагностик, среды и тестов даёт 42.5%, а неудачных попыток — 41.4%; полная история достигает 55.2%.

Экономическая картина согласуется с преимуществом содержательных записей перед бедной памятью. Число наблюдаемых успехов на доллар возрастает с 12.87 до 20.79 уже после добавления изменений и команд. При этом дальнейшее обогащение не улучшает одновременно все показатели. Медианная стоимость не убывает монотонно, и ни один вариант не достигает успешности полной истории.

Это поисковый результат, а не подтверждённая последовательность независимых компонентных эффектов. Конфигурации с меньшей успешностью не продолжили измерять; сравнение поэтому сохраняет статус предварительного отбора. Оно мотивирует исследование содержимого памяти, но не устанавливает, какой отдельный компонент вызвал прирост или сколько выходных токенов удалось сэкономить именно на записи фактов.

6.3 Проверка цены и дополнительной памяти о неудачах

Условный выигрыш по стоимости чувствителен к выбору задач. Для богатой памяти без записей неудач медианное отношение к контролю на собственном подмножестве общих успехов равно 0.92. Если оставить одинаковое подмножество задач, успешных при всех конфигурациях, отношение становится 1.03; для максимального набора записей — 1.17. Снижение издержек относительно памяти со ссылками на файлы сохраняется, но экономия относительно полной истории

неустойчива к этому выбору. Полное сопоставление приведено в приложении B.

Повторное сравнение полных гибридных вариантов проверяет более узкое изменение: добавление записей неудачных попыток (табл. 4). Успешность составляет 38.5% без таких записей и 39.7% с ними; прямое сравнение даёт $p = 0.824$. Заметного дополнительного выигрыша здесь не установлено.

Table 4: Repeated comparison of hybrid state with and without failure records. Output, cache reads and cost are per-execution medians. Token counts are in thousands; table p-values compare each policy with full history.

Approach	Solved rate (%)	Output (ktok)	Cached (ktok)	Median cost ($)	p
Full history	51.7	16.4	62.5	0.0104	–
Hybrid, no failure records	38.5	19.1	10.2	0.0127	4.3×10^{-4}
Hybrid state	39.7	17.3	9.3	0.0120	0.001

Состав памяти оказался содержательным направлением поиска: бедные ссылки и записи результатов действий дают разные результаты. Но автоматическое сохранение фактов не устранило отставание от полной истории. Следующий вопрос касается информации, которая остаётся в самих взаимодействиях и может не попасть ни в семантический блок, ни в набор автоматических полей.

7 Что возвращает недавняя история

7.1 Восстановление успешности

Автоматическое состояние сохраняет выбранные сведения о выполнении, но исходное взаимодействие может содержать больше: точную формулировку ошибки, контекст вывода или последовательность действий. Сохранение недавних записей проверяет, полезна ли эта информация поверх уже имеющегося состояния. В отличие от изменения состава V, здесь модель снова получает часть взаимодействий без их предварительного отбора в поля.

Для состояния модели успешность возрастает с 34.9% без истории до 45.2% при $k = 2$ и 59.0% при $k = 6$ (табл. 5). Разность с полной историей для наибольшего окна сокращается до -2.3 процентного пункта. Бутстрэп-интервал этой разности, $[-7.7, +3.4]$, допускает как отставание, так и небольшое преимущество; близость точечных оценок не устанавливает эквивалентность.

Table 5: Recent-history retention: solved rate and cost per observed success, including failures. Each memory family has its own full-history reference. Differences and exact McNemar tests use that reference. Hybrid cost-per-success values are approximate.

Approach	k	Solved rate (%)	Cost/success ($)	Δ (pp)	p
Model-written memory					
Full history	–	61.3	0.1116	–	–
Model-written state	0	34.9	0.1630	-26.4	2.1×10^{-13}
Model-written state	2	45.2	0.1091	-16.1	9.7×10^{-7}
Model-written state	6	59.0	0.0768	-2.3	0.488
Hybrid memory					
Full history	–	51.7	0.0473	–	–
Hybrid state	0	39.7	0.0468	-12.1	0.001
Hybrid state	2	40.8	0.0462	-10.9	0.001
Hybrid state	6	48.3	0.0501	-3.4	0.392

Для гибридной памяти направление изменений сохраняется: 39.7% без истории, 40.8% при

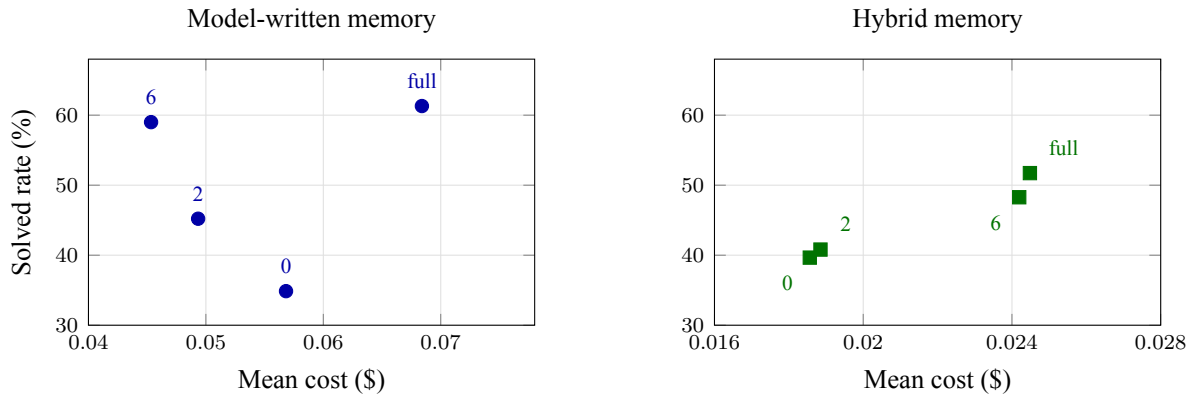


Figure 4: Success–cost trade-offs when retaining history. Labels are the retention setting k; full denotes the full-history reference without a state block. Each panel includes failures in mean cost and uses its own control. Hybrid mean costs use rounded reported totals. The horizontal scales differ.

$k = 2$ и 48.3% при $k = 6$. Наибольшее окно даёт $+8.6$ процентного пункта относительно гибридного состояния без истории ($p = 0.011$). Собственная полная история достигает 51.7%. Автоматические записи, следовательно, не сделали недавние взаимодействия избыточными в исследованной системе.

7.2 Цена восстановления зависит от реализации

Для состояния модели увеличение окна улучшает и наблюдаемую успешность, и стоимость на успех. При $k = 6$ последняя равна \$0.0768 против \$0.1630 без истории и \$0.1116 у полной истории. Сохранение дополнительных записей увеличивает контекст, но итоговый расход определяется всей траекторией, а не только размером одного обращения.

Гибридная система даёт иной компромисс. При добавлении истории $k = 6$ стоимость на успех возрастает с \$0.0468 до \$0.0501; для её контроля она составляет \$0.0473. Более высокая успешность здесь не означает экономии. Рисунок 4 показывает оба случая в координатах качества и средней стоимости.

На общих успехах состояние модели с $k = 6$ сохраняет пиковый контекст 0.81 от контрольного, но стоит 1.15 от контроля. Это не противоречит меньшей стоимости на успех по всей нагрузке: подмножества общих успехов исключают исходы, на которых различается успешность. Выбор представления памяти зависит от того, ограничивает ли систему размер контекста, допустимая доля неудач или совокупный бюджет.

Повторение направления эффекта в обеих реализациях делает сохранение недавней истории наиболее убедительной частичной компенсацией среди проверенных изменений. Возможное объяснение состоит в доступности деталей, не перенесённых в состояние. Однако эксперимент не выделяет эти детали и не сравнивает результат с таким же окном истории без состояния. Дополнительная польза самого блока памяти остаётся отдельным вопросом.

8 Можно ли устранить отставание схемой или протоколом

Полезность недавней истории ещё не означает, что исходные взаимодействия принципиально необходимы. Альтернативное объяснение состоит в неудачном способе записи состояния. Отдельный инструмент обновления отличается от совместного контракта SKILL.state, а общая схема может плохо представлять работу в терминале. Для проверки этих объяснений внутри гибридной системы сравниваются сочетания tool/inline-протоколов и исходной/адаптированной схем (табл. 6).

Адаптированная схема при отдельном инструменте даёт 41.4% против 39.7% у опорного варианта. Наблюдаемое улучшение небольшое, а медианная стоимость возрастает с \$0.0120 до \$0.0137.

Table 6: Protocol and schema changes within hybrid memory, without recent history. Output and cost are per-execution medians; cost per success includes failures. Exact McNemar tests use Tool + Existing as the reference.

Protocol	Schema	Solved rate (%)	Output (ktok)	Median cost ($)	Cost/success ($)	p
Tool	Existing	39.7	17.3	0.0120	0.0468	–
Inline	Existing	35.1	28.6	0.0165	0.0656	0.185
Tool	Task-adapted	41.4	21.7	0.0137	0.0474	0.678
Inline	Task-adapted	28.7	33.1	0.0190	0.1008	0.001

Для восстановления контрольной успешности этого изменения недостаточно.

Inline-протокол с исходной схемой даёт 35.1% и медианную стоимость $0.0165. Совместное изменение протокола и схемы даёт 28.7% при стоимости $0.0190. Вместо ожидаемого устранения двух ограничений наблюдается ухудшение. Описательная разность эффектов inline-протокола между схемами составляет -8.0 процентного пункта. Проверка такого взаимодействия с учётом задач требует совместных исходов, которых нет в доступной агрегированной сводке.

У inline-вариантов больше медианный выход. Совместное формирование обновления и действия поэтому не оказалось бесплатной заменой отдельного инструмента. Расход может включать текст обновлений, исправления формата и другие изменения траектории; доступные агрегаты не разделяют эти составляющие.

Это сравнение исключает простое объяснение, согласно которому достаточно применить именно проверенную схему или совместный формат ответа, чтобы восстановить результат полной истории. Оно не исключает более удачных схем и реализаций. Измеренные изменения отличаются от возврата недавней истории: последний повышает успешность внутри обеих систем памяти, тогда как выбранные изменения представления не дают сопоставимого восстановления.

9 Выводы для устройства памяти агента

9.1 Содержание памяти важнее одного ограничения размера

Результаты разделяют два требования к состоянию: ограничить активный контекст и сохранить сведения для последующих решений. Первое достигается даже бедным представлением. Второе требует выбора содержания. В поисковом сравнении записи изменений и команд соответствуют лучшим результатам, чем список файлов; добавление следующих групп сведений не даёт последовательного выигрыша. Это аргумент в пользу явного проектирования автоматической памяти, но не в пользу её неограниченного расширения.

Разделение ответственности задаёт полезный принцип такого проектирования. Захваченные результаты инструментов могут сохраняться без генерации моделью, а гипотезы и планы остаются семантическим содержанием. К этому необходимо добавить доступ к информации, которую ни один из этих способов не удержал. В испытанных конфигурациях такую роль частично выполняет недавняя история. При этом мы не установили, обеспечивает ли состояние преимущество перед тем же окном взаимодействий без него.

9.2 Компромисс следует выбирать по качеству и затратам

Компактное состояние не является готовой политикой экономии. При полной замене истории оно уменьшает пик контекста, но проигрывает по успешности и может увеличить стоимость сопоставимого успеха. Сохранение недавней истории меняет этот компромисс: для состояния модели оно даёт более низкую стоимость на наблюдаемый успех при меньшем точечном отставании от контроля. Для hybrid восстановление успешности оплачивается дополнительными ресурсами.

Поэтому практический выбор должен исходить из ограничения конкретной системы. Если приоритетом является успешность, потеря истории требует обоснования. Если ограничивает до-

ступное контекстное окно, структурированная память предоставляет измеримую экономию места, но её нужно проверять на способности завершать задачи. Если важен бюджет, необходимо учитывать выход и кеш наряду с входом, а расходы оценивать вместе с неудачами. Одного отношения размеров промпта для такого решения недостаточно.

9.3 Частичная компенсация и непроверенные продолжения

Уже измеренная компенсация состоит в сохранении недавних взаимодействий внутри агента со состоянием. Более богатая автоматическая память представляет перспективное направление по результатам поискового сравнения; проверенные схема и inline-протокол не заменили доступ к истории. Это разные по силе выводы, и они не образуют доказательства превосходства единой новой архитектуры.

Адаптивное переключение между транскриптом и состоянием могло бы учитывать текущую потребность в контексте, а внешний архив с поиском — возвращать наблюдения за пределами недавнего окна. Эти механизмы следуют из различия между активной памятью и сохраняемыми свидетельствами, но в работе не оценивались. Сравнение фиксированных окон не определяет момент переключения, а возврат последних взаимодействий не проверяет восстановление давно удалённых сведений. Такие вопросы требуют собственных контролируемых сравнений.

10 Ограничения исследования

Реализация и отбор. Выводы относятся к конкретным реализациям, а не к структурированной памяти вообще. Полные снимки исполняемого кода, параметры генерации и трассы контекста недоступны для независимого повторения. Для состояния модели описаны правила включения записей, но не установлено независимо, когда они были определены. Описание сбора гибридных результатов оставляет неоднозначность в исключениях, связанных с бюджетом; нельзя исключить зависимость отбора от исхода. Балансировка условий по времени выполнения и маршрутам провайдера не подтверждена.

Поисковое сравнение состава не изолирует отдельные компоненты: измеренные переходы меняют группы записей. Продолжение измерений зависело от предварительной успешности, а часть наблюдений используется также в итоговых гибридных сводках. Поэтому более высокую успешность богатой памяти нельзя трактовать как независимо воспроизведённый причинный эффект делегирования. Имена групп записей описывают устройство представления; полнота и правильность их извлечения отдельно не измерены.

Измерение и статистическая интерпретация. Проверка задачи измеряет выполнение требований бенчмарка, а не качество памяти в отрыве от агента. Значения параметра истории не подтверждают одинаковую семантику сообщений между реализациями. Стоимость по фиксированному тарифу и записанная стоимость гибридных вариантов имеют различную доступность исходного расчёта; оба способа учёта не охватывают все исключённые попытки. Повторные задачи, отбор подмножеств по успешности и множественные поисковые сравнения ограничивают статистические выводы. Проверка на общем подмножестве показывает чувствительность части стоимостных результатов к отбору, но сама также условна на успешном исходе.

Переносимость. Рассмотрено одно сочетание модели и агента на ограниченном наборе задач. Схема адаптировалась на этом же наборе. Результаты могут измениться для других моделей, способов записи состояния, механизмов поиска и задач у границы контекстного окна. Отсутствие контроля с одной лишь недавней историей не позволяет установить дополнительную пользу состояния или взаимодействие двух механизмов. Адаптивное переключение и поиск в архиве не исследованы. Предлагаемые объяснения через потерю деталей, повторное изучение среды и использование кеша не подтверждены причинным анализом траекторий.

11 Связанные работы

SKILL.state [Badhe et al., 2026] заменяет накапливаемую историю изменяемым состоянием исполнения. В исходной работе представлены как синтетические среды, так и интерактивные бенчмарки. Здесь исследуется перенос идеи на работу в терминале с другой моделью, средой исполнения и набором задач. Поэтому отрицательные результаты относятся к этим реализациям и не опровергают исходные измерения.

Для такого сопоставления важно различать заданную извне последовательность событий и траекторию, которую выбирает сам агент. В первом случае можно изучать поведение при увеличении числа событий. В интерактивной задаче программирования политика также влияет на число действий до остановки. Более медленный рост контекста при фиксированной длине не определяет ни стоимость, ни успешность таких траекторий.

Другой способ уменьшить вход — сжатие промпта. LLMLingua [Jiang et al., 2023] применяет отбор токенов с заданным бюджетом. Замена истории состоянием требует поддерживать выбранную информацию в явных полях. Эти подходы по-разному задают представление и сохранение сведений; непосредственное сравнение их качества в данной работе не проводится.

ReAct [Yao et al., 2023] чередует рассуждения и действия с обратной связью среды. Управление контекстом определяет, какие предыдущие записи доступны внутри такого цикла. В наших сравнениях меняется эта информационная политика, но не предполагается, что один объём истории определяет качество рассуждений. Terminal-Bench [Merrill et al., 2026] предоставляет исполняемые задачи и проверки, по которым измеряется итоговое поведение агента.

12 Заключение

SKILL.state предлагает полезное отделение рабочей памяти от растущего транскрипта. Перенос этой идеи в программирующего агента подтверждает возможность уменьшить пиковый контекст, но показывает, что сжатие само по себе не сохраняет успешность и не гарантирует снижения расходов.

Состав состояния меняет результат. В поисковом сравнении автоматические записи изменений и команд заметно лучше бедной памяти со ссылками на файлы; последующее обогащение не даёт монотонного улучшения. В повторных измерениях сохранение недавней истории восстанавливает значительную часть наблюдаемого разрыва и для состояния модели, и для гибридной памяти. Проверенные изменения схемы и протокола такого восстановления не обеспечивают, а экономический эффект истории зависит от реализации.

Практический вывод состоит в разделении решений о памяти: какие наблюдения сохраняет среда исполнения, какую интерпретацию поддерживает модель и какие исходные взаимодействия остаются доступными. Эти решения следует оценивать по успешности и стоимости всего выполнения наряду с размером контекста. Испытанные способы частично компенсируют ограничения замены истории состоянием; их результаты не устанавливают ни универсального преимущества гибридной архитектуры, ни дополнительной пользы состояния перед одним окном недавней истории.

Список литературы

Sanket Badhe, Priyanka Tiwari, and Jonghyun Chung. SKILL.state: Scalable long-horizon agent skills. *arXiv preprint arXiv:2608.26263*, 2026. URL `https://arxiv.org/abs/2608.26263`. v3, 02 Sep 2026.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP*, 2023. doi: 10.18653/v1/2023.emnlp-main.825. URL `https://aclanthology.org/2023.emnlp-main.825/`.

Mike A. Merrill et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL https://arxiv.org/abs/2601.11868. arXiv:2601.11868. Version 2.0 used in this work.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.03629.

A Воспроизводимость и определения показателей

Артефакт анализа содержит записи отдельных выполнений для состояния модели, записи поискового сравнения состава памяти и окончательные агрегированные сводки гибридных подходов. Из этих источников автоматически формируются значения таблиц и числовые утверждения. Первые записи сохраняют исходы и счётчики ресурсов, но не предоставляют полного материала для повторения траекторий. Итоговые гибридные сводки содержат успешность и несовпадающие исходы сопоставлений, достаточные для приведённых разностей и точных проверок Мак-Немара.

Поисковые записи и окончательная гибридная сводка частично используют одни наблюдения. Они не объединяются в увеличенную выборку. Компоненты названы по группам сведений в описании реализации; наличие значения в счётчике ресурсов не считается подтверждением правильной работы каждого извлекателя автоматического состояния.

Для ресурса X условное отношение рассчитывается как

$$\text{median}_{(i,r):\, Y_{iar}=Y_{i0r}=1} \frac{X_{iar}}{X_{i0r}}.$$

Нулевой знаменатель делает отдельное отношение неопределённым; он не заменяется произвольной константой. Входные и кешированные токены учитываются отдельными счётчиками.

Показатели эффективности используют все включённые исходы:

$$\text{Solved/\$} = \frac{\sum Y}{\sum D}, \qquad \text{Solved/Mtok} = \frac{10^6 \sum Y}{\sum (U + K + O)}.$$

Последняя величина описывает логический объём токенов при принятом способе учёта, а не равенство вычислительной работы для кешированных и некешированных токенов. Показатели из окончательных гибридных сводок приближённые, поскольку исходные суммарные затраты округлены. Недоступные суммы токенов и пики контекста не восстанавливаются из отдельных медиан.

B Чувствительность стоимости к подмножеству общих успехов

В поисковом сравнении состава памяти условные отношения можно вычислить на разных подмножествах. Первый способ оставляет для каждого варианта задачи, успешные и при нём, и при полной истории. Второй использует неизменное пересечение успехов всех вариантов и контроля. Таблица 7 сопоставляет эти оценки.

На одинаковом подмножестве богатые наборы записей сохраняют меньшие издержки, чем ссылки на файлы, но их медианная стоимость не ниже контрольной. Эта проверка ограничивает утверждение об экономии относительно полной истории. Она не устраняет отбор по исходу: оба способа рассматривают только определённые успешные выполнения и не оценивают причинный эффект компонента на всей нагрузке.

Table 7: Exploratory resource sensitivity. Variable-subset cost uses each configuration's shared successes with full history. Fixed-subset ratios use the same tasks solved by every configuration and full history. All values are medians of within-task ratios to full history.

Runtime-owned information	Variable subset Cost	Fixed subset Cost	Fixed subset Turns	Fixed subset Output
File references	2.31×	2.50×	1.61×	4.03×
Files, changes, commands	1.20×	1.07×	0.77×	1.56×
+ diagnostics, environment, tests	0.92×	1.03×	0.75×	1.75×
+ failed-attempt records	0.94×	1.17×	0.79×	1.73×

C Стоимость и эффективность всех сохранённых выполнений

Таблица 8 дополняет локальные сравнения единой сводкой медианной и средней стоимости, успехов на доллар и, где доступны суммы, на миллион токенов. В каждой группе используется собственный контроль. Сводка включает неудачи, но не стоимость исключённых попыток.

Table 8: Cost and efficiency over retained executions, including failures. Successes per dollar use aggregate cost and observed successes, not ratios of medians. Token efficiency is reported where cumulative totals are available. Final hybrid efficiencies use rounded reported cost totals.

Approach	Median cost ($)	Mean cost ($)	Solved/$	Solved/Mtok
Model-written memory				
Full history	0.0137	0.0684	8.96	0.421
Model-written state	0.0239	0.0568	6.14	1.245
State + history ($k = 2$)	0.0151	0.0493	9.16	1.964
State + history ($k = 6$)	0.0154	0.0453	13.02	2.718
Hybrid memory (efficiencies approximate)				
Full history	0.0104	0.0245	21.13	–
Hybrid state	0.0120	0.0186	21.36	–
Hybrid + adapted schema	0.0137	0.0196	21.11	–
Hybrid + inline	0.0165	0.0230	15.25	–
Hybrid + inline + adapted schema	0.0190	0.0290	9.92	–
Hybrid + history ($k = 2$)	0.0122	0.0189	21.65	–
Hybrid + history ($k = 6$)	0.0144	0.0242	19.95	–

D Успешность и длина контрольной траектории

Таблица 9 разделяет задачи по медианному числу ходов контроля с полной историей. Все варианты состояния используют одно разбиение. Для состояния без истории отставание больше в подмножестве с длинными контрольными траекториями; сохранение недавней истории повышает успешность в обоих подмножествах. Поскольку разбиение выбрано после эксперимента, а длина траектории зависит и от свойств задачи, эта закономерность не устанавливает причинного влияния длины или точки перехода у границы контекстного окна.

Table 9: Solved rate by the full-history control's median recorded turns T. Differences and exact McNemar tests use full history within the same subset. The grouping is descriptive and held fixed across conditions.

	Control median $T \leq 11$			Control median $T > 11$		
Approach	Solved rate (%)	Δ (pp)	p	Solved rate (%)	Δ (pp)	p
Full history	56.3	–	–	66.7	–	–
Model-written state	39.3	-17.0	6.6×10^{-5}	30.2	-36.5	1.7×10^{-9}
State + history ($k = 2$)	45.2	-11.1	0.011	45.2	-21.4	4.2×10^{-5}
State + history ($k = 6$)	55.6	-0.7	1.000	62.7	-4.0	0.473